AI Lab assignment 7

Tejas Patil
U24AI061

Q1)

```cpp
#include<bits/stdc++.h>
using namespace std;
#define ll long long int

const int N = 8;

// Struct to store simulation results
struct Result {
    int initial_h;
    int final_h;
    int steps;
    bool solved;
};

// Function to generate a random board
vector<int> random_board(mt19937& gen) {
    uniform_int_distribution<> dis(0, N - 1);
    vector<int> board(N);
    for (int i = 0; i < N; ++i) {
        board[i] = dis(gen);
    }
    return board;
}

// Heuristic function: counts pairs of queens attacking each other
int heuristic(const vector<int>& state) {
    int h = 0;
    for (int i = 0; i < N; ++i) {
        for (int j = i + 1; j < N; ++j) {
            // Same row
            if (state[i] == state[j]) {
                h++;
            }
```

```
            // Same diagonal
            if (abs(state[i] - state[j]) == abs(i - j)) {
                h++;
            }
        }
    }
    return h;
}

// Steepest Hill Climbing Algorithm
Result steepest_hill_climbing(vector<int> current) {
    int steps = 0;
    int init_h = heuristic(current);

    while (true) {
        int h_current = heuristic(current);
        if (h_current == 0) return {init_h, 0, steps, true};

        vector<int> best_neigh = current;
        int best_h = h_current;

        // Explore all neighbors
        for (int col = 0; col < N; ++col) {
            int original_row = current[col];
            for (int row = 0; row < N; ++row) {
                if (row == original_row) continue;

                vector<int> neighbor = current;
                neighbor[col] = row;
                int h_neigh = heuristic(neighbor);

                if (h_neigh < best_h) {
                    best_h = h_neigh;
                    best_neigh = neighbor;
                }
            }
        }

        // If no improvement can be made, we are at a local optimum
        if (best_h >= h_current) {
```

```
            return {init_h, h_current, steps, false};
        }

        current = best_neigh;
        steps++;
    }
}

int main() {
    // Seed for randomness
    random_device rd;
    mt19937 gen(rd());

    vector<Result> results;
    int solved_count = 0;

    // Run 50 simulations
    for (int i = 0; i < 50; ++i) {
        vector<int> board = random_board(gen);
        Result res = steepest_hill_climbing(board);
        results.push_back(res);
        if (res.solved) solved_count++;
    }

    // Print Results Header
    cout << setw(10) << "Initial h" << setw(10) << "Final h"
         << setw(10) << "Steps" << setw(10) << "Solved" << endl;
    cout << string(40, '-') << endl;

    // Print each row
    for (const auto& r : results) {
        cout << setw(10) << r.initial_h
             << setw(10) << r.final_h
             << setw(10) << r.steps
             << setw(10) << (r.solved ? "True" : "False") << endl;
    }

    cout << "\nSolved: " << solved_count << " / 50" << endl;

    return 0;
```

```
}
```

Initial h	Final h	Steps	Solved
6	2	3	False
8	2	3	False
7	2	3	False
8	0	4	True
5	2	2	False
5	2	2	False
13	1	4	False
5	1	3	False
7	2	2	False
11	2	3	False
12	1	4	False
8	1	4	False
6	1	2	False
9	0	5	True
5	0	3	True
9	2	3	False
8	1	3	False
10	2	3	False
4	2	2	False
10	1	4	False
5	1	4	False
7	2	3	False
8	2	3	False
6	1	3	False
11	2	4	False
5	2	2	False
8	0	5	True
6	1	3	False
23	0	6	True
7	1	3	False
11	3	3	False
7	1	3	False
10	1	4	False
5	2	2	False
10	2	3	False
5	1	2	False
11	1	5	False
6	0	5	True
7	1	4	False
12	0	4	True
7	2	3	False
7	1	3	False
9	1	4	False
10	1	4	False
5	2	1	False
11	1	4	False
8	2	2	False
8	1	3	False

Q2)

```
#include<bits/stdc++.h>
using namespace std;
#define ll long long int

const int N = 8;
mt19937 gen(random_device{}());

// --- HEURISTIC ---
int heuristic(const vector<int>& board) {
    int conflicts = 0;
    for (int i = 0; i < N; ++i) {
        for (int j = i + 1; j < N; ++j) {
            if (board[i] == board[j] || abs(board[i] - board[j]) == abs(i
- j)) {
                conflicts++;
            }
        }
    }
    return conflicts;
}

// --- GENERATORS ---
vector<int> random_board() {
    uniform_int_distribution<> dis(0, N - 1);
    vector<int> board(N);
    for (int i = 0; i < N; ++i) board[i] = dis(gen);
    return board;
}

vector<vector<int>> get_neighbors(const vector<int>& board) {
    vector<vector<int>> neighbors;
    for (int col = 0; col < N; ++col) {
        for (int row = 0; row < N; ++row) {
            if (board[col] != row) {
                vector<int> next_board = board;
                next_board[col] = row;
                neighbors.push_back(next_board);
            }
        }
```

```
    }
    return neighbors;
}

// --- FIRST CHOICE HILL CLIMBING ---
pair<vector<int>, int> first_choice_hill_climb(vector<int> board) {
    int steps = 0;
    while (true) {
        int current_h = heuristic(board);
        auto neighbors = get_neighbors(board);
        shuffle(neighbors.begin(), neighbors.end(), gen);

        bool found = false;
        for (const auto& n : neighbors) {
            if (heuristic(n) < current_h) {
                board = n;
                steps++;
                found = true;
                break;
            }
        }
        if (!found) return {board, steps};
    }
}

// --- STEEPEST ASCENT (Helper for Random Restart) ---
pair<vector<int>, int> hill_climb(vector<int> board) {
    int steps = 0;
    while (true) {
        auto neighbors = get_neighbors(board);
        vector<int> best = board;
        int best_h = heuristic(board);

        for (const auto& n : neighbors) {
            int h = heuristic(n);
            if (h < best_h) {
                best_h = h;
                best = n;
            }
        }
```

```
        if (best_h >= heuristic(board)) return {board, steps};
        board = best;
        steps++;
    }
}

// --- RANDOM RESTART HILL CLIMBING ---
pair<vector<int>, int> random_restart() {
    int total_steps = 0;
    while (true) {
        vector<int> board = random_board();
        auto [result, steps] = hill_climb(board);
        total_steps += steps;
        if (heuristic(result) == 0) return {result, total_steps};
    }
}

// --- SIMULATED ANNEALING ---
pair<vector<int>, int> simulated_annealing(vector<int> board) {
    double T = 100.0;
    int steps = 0;
    uniform_real_distribution<> dis(0.0, 1.0);
    uniform_int_distribution<> neigh_dis(0, (N * (N - 1)) - 1);

    while (T > 0.1) {
        int current_h = heuristic(board);
        if (current_h == 0) return {board, steps};

        auto neighbors = get_neighbors(board);
        vector<int> next_state = neighbors[neigh_dis(gen)];

        int delta = current_h - heuristic(next_state);

        if (delta > 0 || dis(gen) < exp(delta / T)) {
            board = next_state;
        }

        T *= 0.95;
        steps++;
```

```
    }
    return {board, steps};
}

// --- EXPERIMENT RUNNER ---
void run_experiment(string name) {
    cout << "\n--- " << name << " ---" << endl;
    int success = 0;

    for (int i = 1; i <= 50; ++i) {
        vector<int> board = random_board();
        int initial_h = heuristic(board);
        vector<int> final_board;
        int steps;

        if (name == "First Choice") {
            auto res = first_choice_hill_climb(board);
            final_board = res.first; steps = res.second;
        } else if (name == "Random Restart") {
            auto res = random_restart();
            final_board = res.first; steps = res.second;
        } else {
            auto res = simulated_annealing(board);
            final_board = res.first; steps = res.second;
        }

        int final_h = heuristic(final_board);
        bool solved = (final_h == 0);
        if (solved) success++;

        cout << setw(2) << i << " | Initial h: " << setw(2) << initial_h
             << " | Final h: " << setw(2) << final_h
             << " | Steps: " << setw(4) << steps
             << " | Result: " << (solved ? "Solved" : "Fail") << endl;
    }
    cout << "Success rate: " << success << " / 50" << endl;
}

int main() {
    run_experiment("First Choice");
```

```
    run_experiment("Random Restart");
    run_experiment("Simulated Annealing");
    return 0;
}
```